Universidad de Alcalá

ESCUELA POLITECNICA SUPERIOR

DISEÑO DE SISTEMAS ELECTRÓNICOS DIGITALES

PRÁCTICA 1: ARQUITECTURA DE MULTIPROCESADORES

Alumnos:

Juan Ignacio Figna Pasotti

José Miguel Gimeno Martín

Curso: 2026/2027

ÍNDICE

1 Implementación de una FFT de 512 Puntos con un Núcleo 1
1.1 Objetivo 1
1.2 Resolución de la Práctica 1
1.3 Análisis de Resultados 4
2 Implementación de una FFT de 512 Puntos Mediante la Librería NEON 5
2.1 Objetivo 5
2.2 Resolución de la Práctica 5
2.3 Análisis de Resultados 10
3 Implementación del Funcionamiento de Dos Núcleos con Memoria Compartida 12
3.1 Objetivo 12
3.2 Resolución de la Práctica 12
3.2.1 Desarrollo del Programa Correspondiente a Core 0 15
3.2.2 Desarrollo del Programa Correspondiente a Core 1 21
3.2.3 Sincronización entre los dos Núcleos Mediante Mutex 23
3.3 Análisis de Resultados 24

1 Implementación de una FFT de 512 Puntos con un Núcleo

1.1 Objetivo

El objetivo de esta práctica es implementar y ejecutar una Transformada de Fourier (FFT) de 512 puntos utilizando un único núcleo del procesador ARM de la plataforma Zybo Z7-20. Se busca comprobar la precisión de los resultados comparándolos con unos valores de referencia generados previamente en MATLAB y medir el tiempo de ejecución de la transformada.

1.2 Resolución de la Práctica

Para desarrollar la aplicación se ha programado el archivo principal "main.cc" en C++. En primer lugar, se incluyen las librerías y los archivos necesarios para trabajar con números complejos, acceder a los vectores de prueba y medir el tiempo de ejecución.

```
#include <complex>

#include "DataIn.h"

#include "DataOut_OK.h"

#include "xtime_l.h"

#include <stdio.h>

#include "xil_printf.h"
```

La librería complex permite realizar operaciones con números complejos, necesarios para representar las muestras de entrada y salida de la FFT. Los archivos DataIn.h y DataOut_OK.h contienen, respectivamente, el vector de datos de entrada y los resultados de referencia generados previamente en MATLAB. Por otra parte, xtime_l.h proporciona las funciones necesarias para medir el tiempo de ejecución en el procesador.

A continuación, se definen las variables principales que se utilizan durante el programa:

```
std::complex<float> error[FFT_LENGTH];

int i=0,errorT=0;

float cota=0.69;
```

Mediante FFT_LENGTH se establece que la transformada trabajará con 512 muestras. Se declara el array error, cuyos elementos son números complejos de tipo float, para almacenar las diferencias entre los resultados calculados y los valores de referencia.

La variable i se utiliza como índice para recorrer los vectores, mientras que errorT actúa como contador del número total de muestras que presentan un error superior al permitido. Por último, la

variable “cota” establece la tolerancia máxima admitida para considerar correcto un resultado, fijándose en este caso en 0.69.

Una vez declaradas las variables, se desarrolla la función principal main(). En ella se reserva memoria para almacenar los resultados de la transformada:

XTime tStart, tEnd;

sampleOutX_t dataout[FFT_LENGTH];

Las variables tStart y tEnd permiten almacenar los instantes inicial y final de la ejecución de la FFT. El vector dataout tiene capacidad para almacenar las 512 muestras de salida obtenidas. Su tipo, sampleOutX_t, está definido en los archivos del proyecto.

Ejecución de la FFT y medida del tiempo

La transformada se ejecuta mediante el siguiente fragmento de código:

XTime_GetTime(&tStart);

FlexFFT(DataIn, dataout);

XTime_GetTime(&tEnd);

En primer lugar, se utiliza XTime_GetTime() para registrar el instante inicial de ejecución. A continuación, se llama a la función FlexFFT(), que realiza la transformada de Fourier a partir del vector de entrada DataIn y almacena los resultados en dataout.

Cuando finaliza la ejecución de la función, se vuelve a utilizar XTime_GetTime() para registrar el instante final. De esta manera, la diferencia entre ambos valores permite conocer el tiempo empleado exclusivamente en calcular la FFT, sin incluir el tiempo necesario para comprobar los resultados ni mostrar los mensajes por pantalla.

Para visualizar el tiempo obtenido se utilizan las siguientes instrucciones:

printf("FFT run: %llu clock cycles.\n", 2*(tEnd-tStart));

printf("FFT run:%.4f us.\n", 1.0*(tEnd-tStart)/(COUNTS_PER_SECOND/1000000));

En la primera instrucción se muestra el número de ciclos de reloj empleados. Para ello, se calcula la diferencia entre los instantes inicial y final y se multiplica por dos, teniendo en cuenta que el contador global utilizado en esta plataforma se incrementa cada dos ciclos del reloj del procesador.

En la segunda instrucción se convierte el tiempo medido a microsegundos. Para ello, se utiliza la constante COUNTS_PER_SECOND, que representa la frecuencia de referencia del contador, y se divide entre un millón para obtener el resultado en dicha unidad.

Comprobación de los resultados obtenidos

Una vez ejecutada la FFT, se comprueba que los resultados obtenidos coinciden con los valores de referencia. Para ello, se recorre el vector de salida y se compara cada una de sus 512 muestras con la correspondiente muestra del vector DataOut_OK.

```
for(i=0;i<FFT_LENGTH;i++)
{
   error[i]=dataout[i]-DataOut_OK[i];

   if(abs(error[i]) > cota)
   {
      errorT++;
      printf("ERROR\n");
   }
   else
   {
      printf("NO_ERROR\n");
   }
}
```

El bucle for permite realizar la comprobación de todas las muestras, desde la posición 0 hasta la 511. En cada iteración se calcula la diferencia entre el resultado obtenido por la FFT (dataout[i]) y el valor esperado (DataOut_OK[i]). Esta diferencia se almacena en la posición correspondiente del vector error.

Como los resultados son números complejos, se utiliza abs() para calcular el módulo de la diferencia. De esta forma, se obtiene una medida de la distancia entre el resultado calculado y el resultado de referencia, teniendo en cuenta tanto la parte real como la imaginaria.

A continuación, se compara ese módulo con el valor de tolerancia establecido anteriormente (cota = 0.69). Si el error supera dicho límite, se considera que la muestra no cumple el criterio de precisión, se incrementa el contador errorT y se muestra por pantalla el mensaje ERROR. Por el contrario, si el error se encuentra dentro del límite permitido, se muestra NO_ERROR.

Finalmente, se muestra el número total de muestras que han superado la tolerancia:

```
printf("Errores totales: %d", errorT);
```

Este valor permite evaluar la precisión de la implementación y comprobar si los resultados calculados por la FFT se ajustan a los patrones de referencia proporcionados.

En conclusión, la solución permite ejecutar una FFT de 512 puntos utilizando un único núcleo del procesador, medir su tiempo de ejecución y verificar la precisión de los resultados mediante su comparación con los valores de referencia. Estos resultados servirán como base para comparar posteriormente el rendimiento con las implementaciones de las siguientes prácticas.

1.3 Análisis de Resultados

Tras ejecutar el programa en la plataforma Zybo Z7-20, se obtiene un tiempo de ejecución de la FFT de 3495,2462 µs, equivalente aproximadamente a 3,495 ms, tal y como se ve en la Figura1.

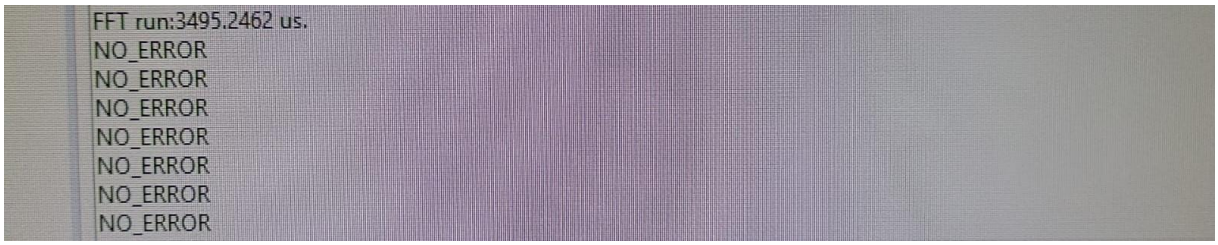


Figura 1. Resultados obtenidos de la práctica 1

Este valor corresponde al tiempo empleado en realizar la transformada de Fourier de 512 puntos utilizando un único núcleo del procesador ARM Cortex-A9.

Una vez finalizada la ejecución, el programa realiza la comprobación de los resultados obtenidos comparándolos con los patrones de referencia generados previamente en MATLAB. Como se observa en la salida por consola, las muestras que aparecen en la imagen muestran el mensaje `NO_ERROR`, lo que indica que la diferencia entre los resultados calculados y los valores de referencia se encuentra dentro de la tolerancia establecida de 0,69.

Por tanto, los resultados mostrados permiten comprobar que la implementación de la FFT proporciona valores que cumplen el criterio de precisión establecido para las muestras verificadas. El tiempo de ejecución obtenido, por su parte, servirá como referencia para evaluar posteriormente las mejoras de rendimiento conseguidas en las siguientes prácticas, tanto mediante la utilización de las instrucciones NEON como mediante el procesamiento paralelo con dos núcleos.

2 Implementación de una FFT de 512 Puntos Mediante la Librería NEON

2.1 Objetivo

El objetivo de esta práctica es acelerar la ejecución de una FFT de 512 puntos utilizando la biblioteca Ne10, que permite aprovechar las instrucciones NEON del procesador ARM Cortex-A9. Se pretende comparar su rendimiento con la implementación de la práctica anterior, midiendo el tiempo medio de ejecución tras realizar 1000 transformadas.

2.2 Resolución de la Práctica

Inclusión de librerías y definición de variables

Una vez configurado el entorno, se desarrolla el programa principal. En primer lugar, se incluyen las librerías necesarias:

```
#include <stdio.h>

#include "xparameters.h"

#include "xil_printf.h"

#include <complex>

using namespace std;

typedef std::complex<float> sampleInX_t;

typedef std::complex<float> sampleOutX_t;

#define FFT_LENGTH 512

#include "DataIn.h"

#include "DataOut_OK.h"

// NEON libraries

#include "Ne10.h"

// Timer libraries

#include "xtime_l.h"
```

Se mantienen los archivos `DataIn.h` y `DataOut_OK.h` de la práctica anterior, ya que contienen los datos de entrada y los patrones de referencia generados previamente en MATLAB.

La principal novedad es la incorporación de `Ne10.h`, que permite acceder a las funciones de la biblioteca Ne10 necesarias para ejecutar la FFT utilizando las instrucciones NEON. También se incluye `xtime_l.h`, que proporciona las funciones necesarias para medir el tiempo de ejecución.

Los tipos `sampleInX_t` y `sampleOutX_t` definen muestras complejas de tipo `float`, aunque en esta implementación los datos se adaptan posteriormente al formato propio de Ne10.

A continuación, se definen las constantes que controlan el tamaño de la transformada y el número de ejecuciones:

#define NFFT FFT_LENGTH

#define TEST_SAMPLES 1000

XTime tStart, tEnd;

La constante `NFFT` toma el valor de `FFT_LENGTH`, por lo que establece que la transformada trabajará con 512 puntos.

Por otra parte, `TEST_SAMPLES` se define con un valor de 1000, indicando que la FFT se ejecutará mil veces para obtener un tiempo medio de ejecución.

Finalmente, se declaran las variables globales `tStart` y `tEnd`, que permitirán registrar los instantes inicial y final de la medición.

Inicialización de la biblioteca NEON y configuración de la FFT

Una vez iniciado el programa, se muestra un mensaje por pantalla indicando el comienzo de la prueba y el tamaño de la transformada:

printf("\n");***

printf(" Launching the FFT test program (%d) ***\n", NFFT);***

printf("\n");***

A continuación, se configura el tamaño de la FFT y se declaran las variables necesarias para gestionar la memoria:

ne10_int32_t fftSize = NFFT;

// Puntero de entrada

ne10_fft_cpx_float32_t *psrc_ddr;

// Puntero de salida

ne10_fft_cpx_float32_t *pdst_ddr;

// Configuración de la FFT

ne10_fft_cfg_float32_t cfg;

// Variable para recorrer los datos

uint32_t i;

La variable `fftSize` indica el número de puntos de la transformada, es decir, 512.

Los punteros `psrc_ddr` y `pdst_ddr` se utilizan para acceder a las posiciones de memoria donde se almacenarán los datos de entrada y los resultados de salida de la FFT, respectivamente. Ambos utilizan el tipo `ne10_fft_cpx_float32_t`, que representa un número complejo en el formato propio de la biblioteca Ne10, con dos componentes: parte real (`r`) y parte imaginaria (`i`).

Por otra parte, `cfg` almacena la configuración de la transformada y los recursos necesarios para ejecutar la FFT mediante NEON.

Antes de utilizar las funciones de la biblioteca, es necesario inicializar Ne10:

```
if (ne10_init() != NE10_OK)
{
   xil_printf("\nFailed to initialise Ne10.\n");
   return 1;
}
```

La función `ne10_init()` prepara la biblioteca para poder utilizar sus operaciones optimizadas. Su resultado se compara con `NE10_OK`, que indica que la inicialización se ha realizado correctamente.

En caso de producirse algún fallo, se muestra un mensaje de error y se finaliza el programa mediante `return 1`, evitando continuar con una biblioteca que no se ha inicializado correctamente.

Después se reserva la memoria necesaria y se obtiene la configuración de la transformada:

cfg = ne10_fft_alloc_c2c_float32_neon(fftSize);

La función `ne10_fft_alloc_c2c_float32_neon()` prepara los recursos necesarios para ejecutar una FFT compleja de 512 puntos utilizando la implementación NEON. Esta configuración contiene, entre otros elementos, los factores necesarios para descomponer la transformada y los coeficientes utilizados durante los cálculos.

Una vez obtenida esta configuración, se inicializan los punteros de entrada y salida:

```
psrc_ddr = (ne10_fft_cpx_float32_t *)
   (cfg->buffer + (sizeof(ne10_fft_cpx_float32_t) * fftSize));
pdst_ddr = (ne10_fft_cpx_float32_t *)
   (psrc_ddr + (sizeof(ne10_fft_cpx_float32_t) * fftSize));
```

La intención de estas instrucciones es establecer las direcciones de memoria donde se almacenarán los datos de entrada y los resultados de salida. El puntero `psrc_ddr` identifica la zona utilizada para introducir las muestras, mientras que `pdst_ddr` identifica la zona destinada a almacenar los resultados de la FFT.

A continuación, se comprueba si la reserva de memoria se ha realizado correctamente:

```
if (cfg == NULL)
{
   xil_printf("\nERROR! Alloc failed.\n");
   return -1;
}
```

Si no ha sido posible reservar los recursos necesarios, la función devuelve un puntero nulo, se muestra un mensaje de error y se finaliza el programa.

Preparación de los datos de entrada

Una vez obtenida la configuración de la FFT mediante la función ne10_fft_alloc_c2c_float32_neon(), se inicializan los punteros que se utilizarán para acceder a los datos de entrada y almacenar los resultados de salida:

```
psrc_ddr = (ne10_fft_cpx_float32_t *)
   (cfg->buffer + (sizeof(ne10_fft_cpx_float32_t) * fftSize));
pdst_ddr = (ne10_fft_cpx_float32_t *)
   (psrc_ddr + (sizeof(ne10_fft_cpx_float32_t) * fftSize));
```

Mediante estas instrucciones se calculan las direcciones de memoria que posteriormente se asignan a los punteros psrc_ddr y pdst_ddr. Para ello, se toma como referencia el buffer asociado a la configuración de la FFT y se aplican desplazamientos calculados a partir del tamaño de cada elemento y del número de puntos de la transformada.

El puntero psrc_ddr se utiliza para acceder a la zona donde se introducen las muestras de entrada, mientras que pdst_ddr se utiliza para indicar dónde se almacenan los resultados obtenidos.

De esta manera, ambos punteros quedan preparados para su utilización en las siguientes etapas del programa: primero, durante la carga de los 512 datos procedentes de MATLAB y, posteriormente, durante la ejecución de la transformada mediante la función ne10_fft_c2c_1d_float32_neon().

Ejecución de la FFT y medida del tiempo medio

Una vez preparados los datos de entrada, se realiza la ejecución de la transformada y se mide el tiempo empleado.

En primer lugar, se registra el instante inicial:

```
XTime_GetTime(&tStart);
```

A continuación, se ejecuta el bucle que realiza las 1000 transformadas:

```
for (int m = 0; m < TEST_SAMPLES; m++) {
  ne10_fft_c2c_1d_float32_neon(
    pdst_ddr,
    psrc_ddr,
    cfg,
    0
  );
}
```

Este bucle repite la ejecución de la FFT un total de 1000 veces, ya que la variable `m` recorre los valores desde 0 hasta 999.

En cada iteración se llama a la función `ne10_fft_c2c_1d_float32_neon()`, que realiza la transformada de Fourier utilizando la implementación optimizada para las instrucciones NEON.

Esta función recibe cuatro parámetros:

- ***`pdst_ddr`: puntero al vector donde se almacenarán los resultados de la transformada.***
- ***`psrc_ddr`: puntero al vector que contiene los datos de entrada.***

- `cfg`: configuración previamente obtenida mediante `ne10_fft_alloc_c2c_float32_neon()`.

- `0`: indica que se desea realizar una FFT directa, no una transformada inversa.

Una vez finalizadas las 1000 transformadas, se registra el instante final:

XTime_GetTime(&tEnd);

De esta forma, la diferencia entre `tEnd` y `tStart` representa el tiempo total empleado en realizar las 1000 ejecuciones de la FFT.

Cálculo y visualización del tiempo medio

Una vez terminadas las ejecuciones, se calcula el tiempo total transcurrido y se convierte a microsegundos:

```
double time_avg = 1000000.0 *
   (double)(tEnd - tStart) /
   (double)COUNTS_PER_SECOND;
```

La diferencia `tEnd - tStart` proporciona el número de unidades de tiempo contabilizadas por el contador. Al dividir entre `COUNTS_PER_SECOND` se obtiene el tiempo en segundos y, al multiplicar por un millón, se convierte a microsegundos.

El resultado almacenado en `time_avg` representa el tiempo total empleado en las 1000 transformadas, expresado en microsegundos.

Para obtener el tiempo medio de una única ejecución, se divide dicho valor entre el número de iteraciones:

```
printf("Time average of execution: %.4f us.\n",
      time_avg / TEST_SAMPLES);
```

De esta manera, se obtiene el tiempo medio que tarda en completarse una FFT de 512 puntos utilizando NEON.

Es importante destacar que la medición comienza justo antes del bucle de ejecuciones y finaliza inmediatamente después. Por tanto, no incluye el tiempo necesario para inicializar la biblioteca, reservar la memoria ni cargar los datos de entrada. Esto permite medir específicamente el tiempo dedicado a realizar las transformadas.

2.3 Análisis de Resultados

Tras ejecutar el programa en la plataforma Zybo Z7-20 donde se utiliza la biblioteca Ne10, se obtiene un tiempo medio de ejecución de la FFT de 25,3369 µs, tal y como se ve en la Figura 2.

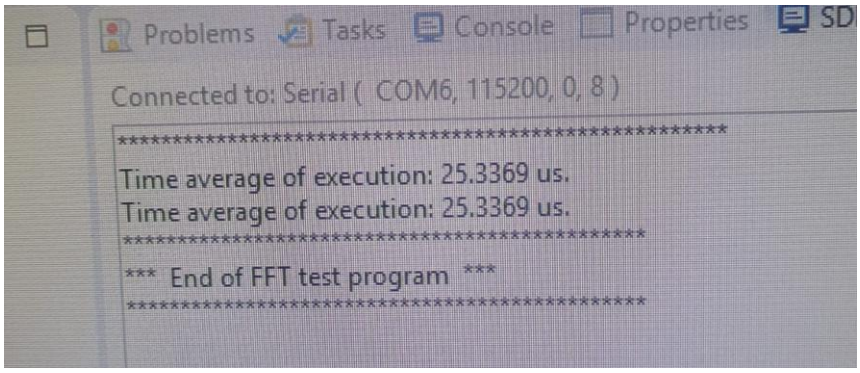


Figura 2. Resultados obtenidos de la práctica 2

Este valor corresponde al promedio de las ejecuciones realizadas para una transformada de 512 puntos, repitiendo el cálculo 1000 veces.

Al comparar este resultado con el obtenido en la primera práctica, donde se registró un tiempo de ejecución de 3495,2462 µs utilizando la implementación convencional mediante `FlexFFT()`, se observa una mejora muy significativa del rendimiento.

En concreto, la implementación con NEON permite realizar la transformada aproximadamente 138 veces más rápido que la implementación anterior.

Por tanto, los resultados obtenidos muestran que la utilización de las instrucciones NEON permite reducir el tiempo necesario para ejecutar la transformada de Fourier de 512 puntos, manteniendo el mismo tamaño de entrada que en la práctica anterior.

3 Implementación del Funcionamiento de Dos Núcleos con Memoria Compartida

3.1 Objetivo

El objetivo de esta práctica es implementar una aplicación que permita trabajar simultáneamente con los dos núcleos ARM Cortex-A9 de la plataforma Zybo Z7-20. Para ello, se realiza una operación de multiplicación sobre un conjunto de 100 datos, dividiendo el trabajo entre ambos núcleos, de manera que cada uno procese 50 elementos.

Se pretende comparar el tiempo de ejecución de esta aplicación con el obtenido al realizar la misma operación utilizando un único núcleo. Además, se utiliza una memoria compartida y un mecanismo de sincronización mediante mutex para coordinar la ejecución de ambos procesadores y garantizar que los resultados se obtienen correctamente.

3.2 Resolución de la Práctica

Configuración de la memoria compartida

Antes de desarrollar los programas, es necesario configurar correctamente la distribución de la memoria DDR en los archivos lscript.ld de ambos proyectos. Según el guion de la práctica, se divide la memoria en tres regiones:

Una región de memoria destinada a Core 0, con origen 0x01000000 y longitud 0x01000000.

Una región de memoria destinada a Core 1, con origen 0x02000000 y longitud 0x01000000.

Una región compartida por ambos núcleos, con origen 0x03000000 y longitud 0x01000000.

La región compartida es especialmente importante, ya que permite que los dos procesadores accedan a los mismos datos, independientemente de cuál de ellos los haya generado o modificado.

Para conseguirlo, las variables que deben utilizar ambos núcleos se sitúan en una sección específica denominada mem_mutex, que se configura en los dos archivos de enlace para que apunte a la misma dirección de memoria compartida.

Además, el guion indica que esta región no debe utilizar memoria caché, ya que ambos procesadores necesitan observar las modificaciones que realiza el otro núcleo. Si cada procesador trabajase con una copia diferente de los datos almacenada en su propia caché, podrían aparecer inconsistencias.

Declaración de variables compartidas en ambos núcleos

En primer lugar, se incluyen las librerías necesarias para trabajar con la plataforma, configurar los atributos de memoria, medir tiempos y utilizar los mecanismos de sincronización:

#include "platform.h"

#include "xil_mmu.h"

#include "xtime_l.h"

#include "mutex_blk.h"

#include <stdio.h>

La librería platform.h proporciona las funciones de inicialización y finalización de la plataforma. xil_mmu.h permite configurar los atributos de acceso a la memoria mediante la unidad de gestión de memoria (MMU), mientras que xtime_l.h proporciona las funciones para medir los tiempos de ejecución.

Por último, mutex_blk.h contiene las declaraciones necesarias para utilizar las funciones lock_mutex() y unlock_mutex(), implementadas en el archivo ensamblador mutex.S.

A continuación, se definen una serie de constantes y macros relacionadas con la configuración de los atributos de memoria:

#define NON_CACHED (0)

#define WB_WA_CACHE (1)

#define WT_CACHE (2)

#define WB_CACHE (3)

#define INNER_SHIFT (2)

#define OUTER_SHIFT (12)

#define TEX_MSB (1 << 14)

#define INNER_NON_CACHED (NON_CACHED << INNER_SHIFT)

#define OUTER_NON_CACHED (TEX_MSB | (NON_CACHED << OUTER_SHIFT))

#define SECTION(attributes) ((attributes) | 0x2)

Estas definiciones permiten establecer las características de acceso a la memoria, incluyendo si se utiliza o no memoria caché. En esta práctica interesa especialmente la configuración de memoria no cacheable, ya que los dos núcleos deben acceder a una misma región de memoria compartida.

También se define la macro SECTION, que permite construir el atributo correspondiente a una sección de memoria en la configuración de la MMU.

Para evitar que el compilador realice determinadas optimizaciones sobre las variables compartidas, se utiliza el siguiente bloque:

#pragma GCC push_options

#pragma GCC optimize ("O0")

Con estas instrucciones se establece un nivel de optimización mínimo para las variables declaradas a continuación. Esto permite evitar que el compilador elimine o modifique determinados accesos a las variables que deben ser compartidas por los dos núcleos.

A continuación, se declaran las variables comunes a ambos procesadores:

```
volatile unsigned int ctrl_mutex
  __attribute__ ((section (".mem_mutex")))
  __attribute__ ((aligned (32))) = unlocked;
#define DATA_LEN 100
volatile unsigned int data[DATA_LEN]
  __attribute__ ((section (".mem_mutex")))
  __attribute__ ((aligned (32)));
volatile unsigned int result[DATA_LEN]
  __attribute__ ((section (".mem_mutex")))
  __attribute__ ((aligned (32)));
volatile unsigned int datos_listos
  __attribute__ ((section (".mem_mutex")))
  __attribute__ ((aligned (32))) = 0;
#pragma GCC pop_options
```

Estas declaraciones son fundamentales para el funcionamiento de la aplicación.

En primer lugar, ctrl_mutex es la variable utilizada como mecanismo de exclusión mutua. Se inicializa con el valor unlocked, indicando que inicialmente no está bloqueada y que puede adquirirse para realizar una operación protegida.

La constante DATA_LEN establece que se van a procesar 100 elementos. El vector data contiene los datos de entrada sobre los que se realizarán las operaciones, mientras que result almacena los resultados obtenidos por cada núcleo.

Por último, datos_listos es una variable de sincronización utilizada para comunicar el estado de ejecución entre los dos procesadores. Su valor inicial es cero y posteriormente irá cambiando para indicar cuándo puede comenzar Core 1 y cuándo ha terminado su trabajo.

Es importante destacar que todas estas variables se declaran como volatile, lo que evita que el compilador suponga que sus valores permanecen constantes durante la ejecución. Esto resulta especialmente necesario porque los dos núcleos pueden modificar o consultar estas variables de forma independiente.

Además, mediante __attribute__((section (".mem_mutex"))) se indica que todas ellas deben situarse en la sección de memoria compartida configurada anteriormente en los archivos del enlazador. El atributo aligned(32) establece un alineamiento de 32 bytes para las variables.

Este mismo bloque de declaraciones aparece tanto en el programa de Core 0 como en el de Core 1, de manera que ambos utilizan las mismas variables y acceden a la misma región física de memoria.

3.2.1 Desarrollo del Programa Correspondiente a Core 0

Core 0 actúa como procesador principal de la aplicación. Se encarga de inicializar los datos, realizar primero el cálculo secuencial, iniciar posteriormente el procesamiento paralelo y esperar a que Core 1 finalice su parte del trabajo.

En primer lugar, dentro de main(), se declaran las variables necesarias:

unsigned int estado;

double tiempo_secuencial;

double tiempo_paralelo;

double comparacion;

int Array[100], i=0, j=0;

XTime tStart, tEnd;

La variable estado permite consultar el estado de sincronización de los procesadores mediante datos_listos.

Las variables tiempo_secuencial y tiempo_paralelo almacenan los tiempos obtenidos al realizar las operaciones con uno y dos núcleos, respectivamente. Por su parte, comparacion permite calcular la relación entre ambos tiempos.

Las variables tStart y tEnd se utilizan para registrar los instantes inicial y final de cada medición.

También se declara el vector local Array, aunque posteriormente las operaciones se realizan sobre el vector compartido data. De esta manera, la comparación entre el cálculo secuencial y el paralelo se realiza utilizando las mismas variables ubicadas en memoria compartida.

Inicialización de la plataforma y configuración de memoria

Antes de comenzar los cálculos, se inicializa la plataforma y se configuran los atributos de la región de memoria compartida:

init_platform();

/* Configurar la memoria compartida */

Xil_SetTlbAttributes(0x03000000,

SECTION(SHAREABLE | NON_CACHED));

La función init_platform() prepara los recursos necesarios para ejecutar la aplicación.

A continuación, Xil_SetTlbAttributes() configura los atributos de memoria de la región que comienza en la dirección 0x03000000, correspondiente a la región compartida definida en el archivo del enlazador.

Se selecciona una configuración de memoria compartible (SHAREABLE) y no cacheable (NON_CACHED). De esta manera, se facilita que ambos núcleos puedan acceder a los mismos datos y observar las modificaciones realizadas por el otro procesador sin depender de copias almacenadas en caché.

Esta misma configuración se realiza también en el programa de Core 1.

Inicialización de los datos

Una vez preparada la memoria, se inicializan los vectores de entrada y salida:

```
for (i = 0; i < DATA_LEN; i++)
{
   Array[i] = i + 1;
   data[i] = i + 1;
   result[i] = 0;
}
```

Este bucle recorre las 100 posiciones de los vectores. En cada posición se introduce un valor consecutivo, comenzando por 1 y terminando en 100.

Por tanto, el vector data contiene los valores que se utilizarán para realizar las multiplicaciones, mientras que el vector result se inicializa a cero para evitar que existan resultados anteriores que puedan interferir en la ejecución.

Aunque también se rellena el vector local Array, los cálculos posteriores se realizan sobre data y result, que se encuentran en la memoria compartida. Esto permite utilizar las mismas condiciones de acceso a memoria en las pruebas secuencial y paralela.

Cálculo utilizando un único núcleo

Antes de ejecutar el procesamiento paralelo, se realiza la operación utilizando exclusivamente Core 0. De esta manera, se obtiene un tiempo de referencia que permitirá evaluar posteriormente la mejora conseguida al utilizar ambos núcleos.

El cálculo se realiza mediante el siguiente fragmento:

```
XTime_GetTime(&tStart);
for (i = 0; i < DATA_LEN; i++)
```

```
{
   result[i] = data[i] * 5;
}
XTime_GetTime(&tEnd);
```

En primer lugar, se registra el instante inicial mediante XTime_GetTime(&tStart). A continuación, se recorre el vector completo y se multiplica cada uno de sus 100 elementos por la constante 5, almacenando los resultados en el vector result.

Una vez finalizado el bucle, se registra el instante final mediante XTime_GetTime(&tEnd). La diferencia entre ambos valores permite conocer el tiempo que ha tardado Core 0 en realizar todas las multiplicaciones.

Para convertir el tiempo obtenido a microsegundos se utiliza la siguiente expresión:

```
tiempo_secuencial =
   1000000.0 * ((double)(tEnd - tStart) /
   (double)COUNTS_PER_SECOND);

printf("Tiempo secuencial medio: %.4f us\r\n",
     tiempo_secuencial);
```

La diferencia entre tEnd y tStart proporciona el número de unidades de tiempo contabilizadas por el contador. Al dividir entre COUNTS_PER_SECOND se obtiene el tiempo en segundos y, al multiplicar por un millón, se convierte a microsegundos.

El resultado se almacena en tiempo_secuencial y se muestra por pantalla.

Preparación para el cálculo paralelo

Una vez finalizado el cálculo secuencial, se vuelven a inicializar los resultados:

```
for (i = 0; i < DATA_LEN; i++)
{
   result[i] = 0;
}
```

Esto permite comenzar la siguiente prueba sin conservar los resultados de la ejecución anterior.

A continuación, se introduce una espera de diez segundos:

```
sleep(10);
```

Esta espera permite que Core 1 termine de inicializarse antes de comenzar la medición paralela, evitando los problemas que se habían observado con el contador utilizado por XTime_GetTime() cuando el segundo núcleo se inicializaba posteriormente.

Inicio del procesamiento paralelo y sincronización

Una vez preparados los datos, se inicia el trabajo de ambos procesadores:

```
printf("Empieza core 0 y core 1\r\n");

lock_mutex(&ctrl_mutex);

datos_listos = 1;

unlock_mutex(&ctrl_mutex);

printf("Mutex Ejecutado\r\n");
```

En primer lugar, Core 0 adquiere el mutex mediante lock_mutex(&ctrl_mutex). De esta forma, bloquea temporalmente el acceso protegido a la variable de sincronización.

A continuación, modifica datos_listos y le asigna el valor 1. Este valor indica a Core 1 que los datos ya están preparados y que puede comenzar a realizar su parte del cálculo.

Finalmente, se libera el mutex mediante unlock_mutex(&ctrl_mutex), permitiendo que el otro procesador pueda acceder al mecanismo de sincronización.

Por tanto, el valor de datos_listos funciona como una señal de comunicación entre los dos núcleos: cuando vale 0, Core 1 todavía no debe empezar; cuando pasa a valer 1, puede comenzar el procesamiento.

Cálculo de Core 0 dentro de la ejecución paralela

Una vez comunicada la señal de inicio, Core 0 realiza la primera mitad de las operaciones:

```
XTime_GetTime(&tStart);

/* Core 0 calcula los primeros 50 datos */

for (i = 0; i < DATA_LEN / 2; i++)

{

   result[i] = data[i] * 5;

}
```

Al igual que en la prueba secuencial, se registra el instante inicial de ejecución. Sin embargo, esta vez el bucle únicamente recorre las primeras 50 posiciones del vector.

Como DATA_LEN vale 100, la expresión DATA_LEN / 2 establece que Core 0 debe procesar los elementos comprendidos entre las posiciones 0 y 49.

Mientras Core 0 realiza estas operaciones, Core 1 ejecuta simultáneamente la segunda mitad del cálculo. De esta manera, ambos procesadores trabajan en paralelo sobre regiones diferentes del vector result, evitando que escriban simultáneamente en las mismas posiciones.

Espera a que Core 1 termine

Una vez que Core 0 ha completado sus 50 multiplicaciones, debe esperar a que Core 1 finalice su parte del trabajo.

Para ello, se utiliza el siguiente bucle:

```
do
{
   estado = datos_listos;

} while (estado != 2);
```

En este caso, Core 0 consulta continuamente el valor de datos_listos y lo almacena en la variable estado.

Mientras el valor sea diferente de 2, el procesador permanece esperando. Cuando Core 1 termina las operaciones que tiene asignadas, modifica datos_listos y establece su valor en 2.

De esta manera, Core 0 detecta que el segundo procesador ha terminado y puede continuar con la ejecución.

Este mecanismo se conoce como espera activa (busy waiting), ya que el procesador permanece ejecutando un bucle de comprobación hasta que se cumple la condición establecida.

Una vez recibida la señal de finalización, se registra el instante final:

```
XTime_GetTime(&tEnd);
tiempo_paralelo =
   1000000.0 * ((double)(tEnd - tStart) /
   (double)COUNTS_PER_SECOND);
printf("Tiempo paralelo medio: %.4f us\r\n",
    tiempo_paralelo);
```

De esta manera, se calcula el tiempo transcurrido desde el inicio de la medición paralela hasta que ambos procesadores han terminado su trabajo.

También se muestran los valores inicial y final del contador:

```
printf("Ticks start: %f\r\n", (double)tStart);
```

printf("Ticks end: %f\r\n", (double)tEnd);

Estos valores permiten visualizar las marcas de tiempo utilizadas para calcular la duración de la ejecución.

Comprobación de los resultados

Una vez finalizado el procesamiento paralelo, se comprueba que las multiplicaciones realizadas por ambos procesadores sean correctas:

```
for (i = 45; i < 55; i++)
{
  if (result[i] != data[i] * 5)
  {
    printf("Error en el dato %d\r\n", i);
    break;
  }
}
```

Este bucle comprueba los resultados comprendidos entre las posiciones 45 y 54 del vector.

Se selecciona esta zona porque permite comprobar valores correspondientes a los dos núcleos y, especialmente, los elementos próximos al punto en el que se divide el trabajo. Recordemos que Core 0 procesa las posiciones de 0 a 49, mientras que Core 1 procesa las posiciones de 50 a 99.

En cada iteración se compara el resultado almacenado en result[i] con el valor esperado, que se obtiene multiplicando el dato original por 5.

Si existe alguna diferencia, se muestra un mensaje indicando la posición en la que se ha detectado el error y se interrumpe el bucle mediante break.

A continuación, el programa incluye la siguiente comprobación:

```
if (i == DATA_LEN)
{
  printf("Todos los resultados son correctos\r\n");
}
```

Su intención es mostrar un mensaje cuando todas las comprobaciones se hayan realizado correctamente. No obstante, en esta versión existe un detalle que conviene corregir: como el bucle únicamente recorre las posiciones de 45 a 54, si termina sin errores el valor de i será 55 y no 100. Por tanto, esta condición no se cumple.

Para que el mensaje se muestre correctamente manteniendo la comprobación de esas diez posiciones, la condición debería ser if (i == 55).

Comparación de los tiempos de ejecución

Finalmente, se calcula la relación entre el tiempo secuencial y el tiempo paralelo:

```
if (tiempo_paralelo > 0.0)
{
   comparacion = tiempo_secuencial / tiempo_paralelo;

   printf("Comparacion: %.2f veces\r\n", comparacion);
}
```

Se divide el tiempo obtenido utilizando un único núcleo entre el tiempo obtenido utilizando los dos núcleos.

El resultado permite conocer la relación de rendimiento entre ambas ejecuciones. Si el valor es superior a 1, significa que la ejecución paralela ha tardado menos tiempo que la secuencial. Por ejemplo, un resultado de 2 indicaría que el procesamiento paralelo ha sido dos veces más rápido en esta prueba.

Finalmente, se utiliza cleanup_platform() para liberar y finalizar los recursos asociados a la plataforma.

3.2.2 Desarrollo del Programa Correspondiente a Core 1

El segundo programa se ejecuta en Core 1 y tiene como finalidad realizar la segunda mitad de las operaciones asignadas. A diferencia de Core 0, este procesador no realiza la comparación secuencial ni mide los tiempos de ejecución. Su función principal es esperar la señal de inicio, procesar los datos que le corresponden y comunicar al primer núcleo que ha terminado.

Al igual que en Core 0, se incluyen las librerías necesarias:

```
#include "platform.h"
#include "xil_mmu.h"
#include "mutex_blk.h"
#include <stdio.h>
```

Se mantienen las librerías de la plataforma, de configuración de memoria y de sincronización. En este caso, no se utiliza xtime_l.h porque Core 1 no realiza directamente las mediciones de tiempo.

También se declaran las mismas variables compartidas que en el programa de Core 0, situadas en la sección mem_mutex, para garantizar que ambos procesadores accedan a los mismos datos.

Inicialización y configuración de memoria

Dentro de la función main(), se inicializa la plataforma y se configura la región compartida:

```
init_platform();

Xil_SetTlbAttributes(0x03000000,

   SECTION(SHAREABLE | NON_CACHED));
```

De esta manera, Core 1 utiliza la misma configuración de memoria que Core 0, permitiendo acceder a los vectores data y result, así como a las variables de sincronización.

Espera de la señal de inicio

Una vez inicializado, Core 1 no comienza inmediatamente a realizar las operaciones, sino que espera a que Core 0 indique que los datos están preparados:

```
while (datos_listos != 1)

{

   /* Espera activa */

}
```

Este bucle comprueba continuamente el valor de la variable compartida datos_listos.

Mientras su valor sea cero, Core 1 permanece esperando. Cuando Core 0 modifica la variable y establece su valor en 1, Core 1 detecta el cambio y continúa con el programa.

De esta manera, se evita que el segundo procesador empiece a trabajar antes de que el primero haya inicializado los datos y preparado la ejecución paralela.

Cálculo de la segunda mitad de los datos

Una vez recibida la señal de inicio, Core 1 realiza las operaciones correspondientes a la segunda mitad del vector:

```
for (i = DATA_LEN / 2; i < DATA_LEN; i++)

{

   result[i] = data[i] * 5;

}
```

Como DATA_LEN vale 100, el bucle comienza en la posición 50 y termina en la posición 99.

En consecuencia, Core 1 se encarga de multiplicar por 5 los últimos 50 elementos del vector data y almacenar los resultados en las posiciones correspondientes de result.

Mientras tanto, Core 0 realiza la misma operación sobre los primeros 50 elementos. De esta manera, ambos procesadores trabajan simultáneamente y se reparten el conjunto de operaciones.

Es importante destacar que cada núcleo modifica una parte diferente del vector de salida. Por tanto, no existe solapamiento entre las posiciones que escriben ambos procesadores.

Comunicación de la finalización del trabajo

Cuando Core 1 termina sus operaciones, debe informar a Core 0 para que este pueda finalizar la medición del tiempo paralelo.

Para ello, se utiliza el siguiente fragmento:

```
lock_mutex((void *)&ctrl_mutex);

datos_listos = 2;

unlock_mutex((void *)&ctrl_mutex);
```

En primer lugar, Core 1 adquiere el mutex mediante lock_mutex(). A continuación, cambia el valor de datos_listos a 2 para indicar que su parte del cálculo ha finalizado.

Finalmente, libera el mutex con unlock_mutex(), permitiendo que Core 0 pueda continuar con la ejecución.

De esta manera, el valor 2 actúa como señal de finalización. Cuando Core 0 detecta este valor en su bucle de espera, sabe que Core 1 ha terminado de procesar sus 50 elementos y que el cálculo paralelo está completo.

Por último, se mantiene el segundo procesador activo mediante un bucle infinito:

```
while (1)

{

  /* Espera */

}
```

Esto evita que Core 1 finalice su aplicación inmediatamente después de completar las operaciones y permite mantenerlo ejecutándose mientras Core 0 continúa con las comprobaciones y la comparación de tiempos.

3.2.3 Sincronización entre los dos Núcleos Mediante Mutex

Uno de los aspectos importantes de esta práctica es la coordinación entre ambos procesadores. Aunque cada núcleo realiza operaciones sobre una parte diferente del vector, necesitan intercambiar información para saber cuándo pueden comenzar y cuándo han terminado.

Para ello, se utilizan dos elementos: la variable compartida datos_listos y el mutex ctrl_mutex.

La variable datos_listos permite conocer el estado de la aplicación mediante tres valores:

- Valor 0: los datos todavía no están preparados y Core 1 debe esperar.
- Valor 1: Core 0 ha preparado los datos y autoriza a Core 1 a comenzar.
- Valor 2: Core 1 ha terminado las operaciones que tenía asignadas y comunica a Core 0 que puede finalizar la ejecución paralela.

Por otra parte, lock_mutex() y unlock_mutex() permiten adquirir y liberar el mutex para proteger las modificaciones de la variable de sincronización.

El funcionamiento conjunto de ambos mecanismos es el siguiente: Core 0 adquiere el mutex, establece datos_listos = 1 y libera el bloqueo. Core 1 detecta el cambio y comienza sus operaciones. Una vez terminadas, adquiere el mutex, establece datos_listos = 2 y lo libera. Finalmente, Core 0 detecta este último cambio y continúa con la ejecución.

De esta manera, se establece una comunicación entre ambos procesadores sin necesidad de que los dos ejecuten simultáneamente las mismas operaciones.

En esta implementación, el mutex se utiliza principalmente para proteger las modificaciones de la variable de sincronización. No es necesario bloquear cada multiplicación, ya que cada núcleo trabaja sobre posiciones diferentes del vector de resultados.

3.3 Análisis de Resultados

Tras ejecutar los dos programas en la plataforma Zybo Z7-20, se obtienen los tiempos de ejecución que se muestran en la Figura 3, correspondientes al procesamiento secuencial y paralelo de los 100 datos.

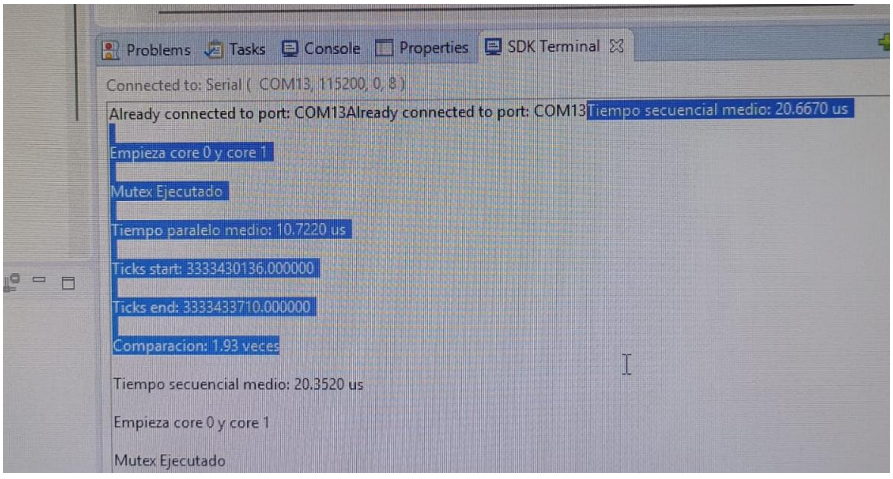


Figura 3. Resultados obtenidos de la práctica 3

En primer lugar, utilizando únicamente Core 0, se obtiene un tiempo de ejecución de 20,3520 µs, necesario para realizar las 100 multiplicaciones por la constante 5. Este valor constituye la referencia para evaluar posteriormente el rendimiento de la ejecución con dos núcleos.

A continuación, se realiza el mismo cálculo distribuyendo el trabajo entre Core 0 y Core 1, asignando 50 elementos a cada procesador. En este caso, se obtiene un tiempo de ejecución de 10,7220 µs.

Este tiempo comprende tanto las operaciones realizadas por ambos núcleos como la espera necesaria para recibir la señal de finalización de Core 1.

Al comparar ambos resultados, se observa que la ejecución paralela permite reducir considerablemente el tiempo empleado en realizar las operaciones. A partir de los tiempos obtenidos, la implementación con dos núcleos consigue aproximadamente 1,90 veces más rendimiento que la ejecución secuencial.

Por tanto, los resultados obtenidos permiten comprobar que la utilización de dos núcleos y la distribución del trabajo entre ambos proporciona una mejora de rendimiento respecto a la ejecución con un único procesador. Además, se verifica la utilidad de la memoria compartida y del mecanismo de sincronización mediante mutex para coordinar las operaciones y permitir que ambos núcleos trabajen simultáneamente sobre diferentes partes de los datos.